

Aswin Krishnamoorthy

Staff Software Engineer / Full-Stack SME | Architecture & System Design

+91 88705 94806 | winnyboy5@gmail.com | Chennai, Tamil Nadu, India

linkedin.com/in/aswin-krishnamoorthy | github.com/winnyboy5

SUMMARY

I've spent 12+ years building web and workflow platforms from the ground up — frontend, backend, databases, deployment, and cloud infrastructure. I'm a hands-on generalist who bridges system architecture with daily production coding: designing API contracts, caching strategies, and compute trade-offs (serverless vs. containers), while picking up unfamiliar technologies fast — like architecting a storage engine in Rust using AI-assisted workflows. I believe the best architecture decisions come from engineers who still build daily and understand the implementation pain firsthand.

SKILLS

- **Core & Languages:** JavaScript (ES6+), TypeScript, Python, PHP, SQL, Rust (Applied), Node.js, Vue.js, Flask, FastAPI, Laravel
- **Databases:** PostgreSQL, MongoDB, MySQL
- **Cloud & DevOps:** Azure (Functions, Blob Storage), Docker, Git, CI/CD (GitHub Actions)
- **Architecture & Design:** System design, RESTful APIs, microservices, caching (in-memory + CDN), API versioning, event-driven patterns
- **AI & Productivity:** AI-assisted development (Claude, LLM coding assistants), agentic workflows, prompt engineering

WORK EXPERIENCE

WPP Production / WPP Enterprise Technology, Chennai, India

June 2019 – Present

Sr. Full-Stack Developer — SME

- Our workflow module had a slow data-fetching layer — offset pagination was falling apart at scale, and we were getting production incidents from slow endpoints. I redesigned it with query result caching (TTL-based invalidation) and switched to cursor-based pagination, which fixed both problems.
- When we needed to decide between serverless (Azure Functions) and containers for a workflow module, I benchmarked both. Serverless won on cost and auto-scaling, despite cold-start latency concerns. I led that migration end-to-end.
- Designed the API contract and versioning strategy for our workflow platform. Three different frontend apps consume it, so backward compatibility wasn't optional — I had to make it work across all of them.
- Took a legacy monolithic UI and broke it into a reusable component library (Vue/Angular) with a shared design token system. Teams stopped building the same components differently after that.
- Redesigned our CI/CD pipeline to use staged deployments with automated smoke tests. Before that, we had manual deployment checklists and too many failed production deploys.
- Mentored 4 junior developers — mostly through code reviews, but also by defining component and API structure standards that the whole team adopted.

Cognizant Technology Solutions, Coimbatore, TN, India

April 2016 – May 2019

Frontend Developer

- Built a shared JavaScript utility library that two client projects (Insurance and Healthcare) both relied on. Having one place for error handling and data transformation patterns kept things consistent and made onboarding faster.
- Set up component contracts and automated visual regression testing. We had UI bugs that kept coming back sprint after sprint — this stopped that cycle.

Spouseup Technologies, Coimbatore, TN, India

March 2015 – November 2015

Web Developer

- The site was slow, so I dug into the queries — found joins that could be restructured, added strategic indexes, and optimized how assets loaded on mobile. Page load times dropped significantly.

iProtecs Solutions, Coimbatore, TN, India

November 2013 – February 2015

UI Developer

- Sat with server engineers, watched how they used the internal tools, and redesigned multi-step workflows into single-screen interfaces. Fewer clicks, fewer errors, less training needed.

Laswa Technologies, Coimbatore, TN, India

October 2012 – October 2013

Web Developer

- Built checkout flows for e-commerce sites and CMS features. Also handled structured data markup and page speed work — the kind of thing that doesn't feel glamorous but directly moved organic search numbers.

SELECTED PROJECTS

MediaGit — Media Version Control Engine (Rust)

- Git doesn't handle large binary files well — video, images, design assets. I designed a content-addressable storage model that solves the versioning, diffing, and deduplication problems for these kinds of files.
- I'd never written Rust before this project. Used AI-assisted development for the language-specific parts, but the architecture, the module boundaries, and every design decision were mine.
- Set up a CI/CD pipeline (GitHub Actions) with cross-platform builds, automated tests, and release artifact generation.
- Built the core CLI: versioning, diffing, restoring media files — designed for teams that manage large video and image libraries.

EDUCATION

Bachelor of Engineering in Computer Science (B.E.)

EASA College of Engineering & Technology — ANNA University of Technology, TN, India

Graduation Year: 2012